

Chapter 10: Arrays in Java

Personal Notes

Until now, you have learned variables (store one value), loops (repeat logic), conditionals (make decisions), and methods (reuse logic). Now comes one of the most important concepts in all of programming.

Arrays let you store and process many values at once — without creating 100 separate variables. This single idea is the foundation of searching, sorting, matrices, and almost every DSA problem you will ever solve.

1. What is an Array?

An array is a collection of elements of the SAME data type, stored in continuous memory locations. You access each element using an index — a number that says "give me the element at position N."

Imagine 5 lockers in a row, numbered 0, 1, 2, 3, 4. Each locker holds one item. That row of lockers is an array.

Example

```
int[] scores = {85, 90, 78, 92, 88};
```

Memory Representation

Index:	0	1	2	3	4
Value:	85	90	78	92	88

Important: Array indexing in Java starts from 0, NOT 1. The first element is at index 0, and the last element is at index (length - 1). This is one of the most common beginner mistakes.

2. Why Arrays Matter

Arrays are everywhere in real programming. You will use them in:

- DSA problems (almost every single one)
- Searching and sorting algorithms
- Storing multiple test cases or records
- Matrices and grids (2D arrays)
- Image processing (an image is just a 2D array of pixels)

- Analytics and data processing

Without arrays, you would need 100 separate variables to store 100 values. With arrays, you store them all in one place and process them with a single loop.

3. 1D Array Basics

3.1 Declaration

Declaration just tells Java that a variable will hold an array. It does not create the array yet.

```
int[] arr;
```

3.2 Initialization

Initialization actually creates the array in memory. There are two common ways.

Way 1: Fixed size, default values

```
int[] arr = new int[5];  
// creates an array of 5 ints, all set to 0
```

Way 2: Direct values

```
int[] arr = {1, 2, 3, 4, 5};  
// array of size 5 with given values
```

3.3 Accessing Elements

```
int[] arr = {10, 20, 30, 40, 50};  
  
System.out.println(arr[0]); // 10  
System.out.println(arr[2]); // 30  
System.out.println(arr[4]); // 50  
  
arr[1] = 99; // update element at index 1  
System.out.println(arr[1]); // 99
```

3.4 Length of an Array

Use `arr.length` to get the size. Note: it is a property, NOT a method, so there are no parentheses.

```
int[] arr = {10, 20, 30};  
System.out.println(arr.length); // 3
```

3.5 Traversal (Looping through the array)

```
for (int i = 0; i < arr.length; i++) {  
    System.out.println(arr[i]);  
}
```

```
}
```

You can also use the enhanced for-loop (for-each), which is cleaner when you don't need the index:

```
for (int value : arr) {  
    System.out.println(value);  
}
```

Common bug: Using `i <= arr.length` instead of `i < arr.length` will throw `ArrayIndexOutOfBoundsException`. The last valid index is `arr.length - 1`, not `arr.length`.

4. 1D Array Problems

Problem 1: Sum of Elements

```
int[] arr = {10, 20, 30, 40, 50};  
int sum = 0;  
  
for (int i = 0; i < arr.length; i++) {  
    sum += arr[i];  
}  
  
System.out.println(sum);    // 150
```

Problem 2: Maximum Element

```
int[] arr = {12, 45, 7, 89, 23};  
int max = arr[0];  
  
for (int i = 1; i < arr.length; i++) {  
    if (arr[i] > max) {  
        max = arr[i];  
    }  
}  
  
System.out.println("Max: " + max);    // 89
```

Start by assuming the first element is the maximum, then compare each remaining element against it.

Problem 3: Minimum Element

Exactly the same idea as Maximum, just flip the comparison:

```
int min = arr[0];  
  
for (int i = 1; i < arr.length; i++) {  
    if (arr[i] < min) {  
        min = arr[i];  
    }  
}
```

Problem 4: Reverse an Array (Two-Pointer Technique)

Use two pointers — one at the start, one at the end — and swap them while moving toward each other.

```
int[] arr = {1, 2, 3, 4, 5};
int start = 0, end = arr.length - 1;

while (start < end) {
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    start++;
    end--;
}
// arr is now {5, 4, 3, 2, 1}
```

Visual

```
Before:    [1, 2, 3, 4, 5]
           ^         ^
           start     end

Swap → [5, 2, 3, 4, 1]
        start++, end--

        [5, 2, 3, 4, 1]
           ^         ^

Swap → [5, 4, 3, 2, 1]
        start++, end--    (now start >= end → stop)

Final:    [5, 4, 3, 2, 1]
```

Problem 5: Linear Search

Check every element one by one until you find the target. Returns true if found, false otherwise.

```
public static boolean linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return true;
        }
    }
    return false;
}
```

5. What is a 2D Array?

A 2D array is an array of arrays. Visually, it looks like a table — rows and columns. Each cell is accessed using two indices: `matrix[row][column]`.

Example: a 3×3 matrix

	col 0	col 1	col 2
row 0	1	2	3
row 1	4	5	6
row 2	7	8	9


```
matrix[0][0] = 1    matrix[1][2] = 6  
matrix[2][1] = 8    matrix[2][2] = 9
```

5.1 Declaration

```
int[][] matrix;
```

5.2 Initialization

Way 1: Fixed size

```
int[][] matrix = new int[3][3];  
// 3 rows × 3 columns, all 0 by default
```

Way 2: Direct values

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

5.3 Accessing Elements

```
System.out.println(matrix[1][2]);    // 6  
System.out.println(matrix[0][0]);    // 1  
  
matrix[2][2] = 99;                   // update bottom-right cell
```

Reading the indices: `matrix[i][j]` means row `i`, column `j`. Row comes first, column second. Mixing this up is the #1 cause of bugs in matrix problems.

6. Traversing a 2D Array

To visit every cell, you need nested loops — outer loop for rows, inner loop for columns.

```
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

Two important things to know:

- `matrix.length` → number of rows
- `matrix[i].length` → number of columns in row `i`

Tip: Outer loop = rows, inner loop = columns. If you ever need to traverse column-by-column instead, just swap the loops so `j` is outer and `i` is inner.

7. 2D Array Problems

Problem 1: Print a Matrix

```
public class Main {
    public static void main(String[] args) {

        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6}
        };

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
1 2 3
4 5 6
```

Problem 2: Row-wise Sum

For each row, add up all its elements and print the total.

```
for (int i = 0; i < matrix.length; i++) {
```

```

    int sum = 0;

    for (int j = 0; j < matrix[i].length; j++) {
        sum += matrix[i][j];
    }

    System.out.println("Row " + i + " sum = " + sum);
}

```

Each row is treated like a small 1D array. Reset sum to 0 at the start of every row.

Problem 3: Column-wise Sum

Same idea as row-wise, but flip the loops — outer loop is columns now, inner loop is rows.

```

for (int j = 0; j < matrix[0].length; j++) {

    int sum = 0;

    for (int i = 0; i < matrix.length; i++) {
        sum += matrix[i][j];
    }

    System.out.println("Column " + j + " sum = " + sum);
}

```

Key insight: When the outer loop is j (columns), you visit cells column-by-column instead of row-by-row. The data is the same — only the order you visit it changes.

Problem 4: Find Maximum Element in Matrix

```

int max = matrix[0][0];

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        if (matrix[i][j] > max) {
            max = matrix[i][j];
        }
    }
}

System.out.println("Max: " + max);

```

Same idea as finding max in a 1D array, just with nested loops.

Problem 5: Transpose of a Matrix

Transpose means flipping rows into columns. The element at $[i][j]$ moves to position $[j][i]$.

Example

Original:	Transpose:
+---+---+---+	+---+---+
1 2 3	1 4
+---+---+---+	+---+---+
4 5 6	2 5
+---+---+---+	+---+---+
	3 6
	+---+---+

Code (just print the transpose)

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};

for (int j = 0; j < matrix[0].length; j++) {
    for (int i = 0; i < matrix.length; i++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

Key insight: To transpose, just swap the row and column indices when accessing: `matrix[i][j]` becomes `matrix[j][i]`. Walk columns in the outer loop and rows in the inner loop.

8. Assignment Problems (2D Arrays)

Try these on your own to lock in 2D array thinking.

1. Print only the diagonal elements (`matrix[i][i]` for $i = 0$ to $n-1$).
2. Calculate the sum of the main diagonal.
3. Search for a given element in a matrix (return true/false).
4. Check if a matrix is symmetric — that is, `matrix[i][j] == matrix[j][i]` for every i, j .
5. Print a matrix in spiral order (advanced).
6. Find the largest element in each row.
7. Rotate the matrix 90° clockwise.

Tip for diagonal problems: For the main diagonal, use `matrix[i][i]`. For the anti-diagonal (top-right to bottom-left), use `matrix[i][n-1-i]` where n is the size.

9. Common Mistakes in Arrays

Mistake 1: Wrong index usage

```
arr[arr.length]    // ERROR – last valid index is arr.length - 1
```

Mistake 2: Using <= instead of <

```
for (int i = 0; i <= arr.length; i++) { ... }    // ERROR
for (int i = 0; i < arr.length; i++) { ... }    // correct
```

Mistake 3: Forgetting nested loops in 2D

A 2D array NEEDS two loops — one for rows, one for columns. A single loop will not work.

Mistake 4: Confusing rows and columns

matrix[i][j] = row i, column j. Always row first. Saying matrix[j][i] when you meant matrix[i][j] is the most common matrix bug.

Mistake 5: Using length() instead of length

```
arr.length()    // ERROR – length is a property for arrays, not a method
arr.length      // correct
str.length()    // correct only for Strings
```

Mistake 6: Forgetting to reset sum/count inside loops

In row-wise problems, declare sum = 0 INSIDE the outer loop, not before it — otherwise it keeps adding across rows.

10. 1D vs 2D Arrays

Feature	1D Array	2D Array
Structure	Single line of values	Table — rows and columns
Access	arr[i]	matrix[i][j]
Loop	Single loop	Nested loops (rows × cols)
Length	arr.length	matrix.length (rows), matrix[i].length (cols)
Memory	One row of cells	Multiple rows stacked
Used for	Lists, scores, basic data	Matrices, grids, images, game boards

11. Quick Reference

Syntax	Meaning
<code>int[] arr;</code>	Declare an int array (no memory yet)
<code>new int[5]</code>	Create empty array of size 5 (defaults to 0)
<code>{1, 2, 3}</code>	Initialize with values directly
<code>arr[i]</code>	Access or update element at index i
<code>arr.length</code>	Get total number of elements (no parentheses)
<code>int[][] m = new int[3][4]</code>	3 rows × 4 columns, all 0
<code>m[i][j]</code>	Element at row i, column j
<code>m.length</code>	Number of rows
<code>m[i].length</code>	Number of columns in row i
<code>for (int v : arr)</code>	For-each loop, no index needed

12. Why Arrays Matter for Interviews

Arrays are the starting point of DSA. Almost every advanced topic in coding interviews builds on top of arrays:

- Searching algorithms (Linear Search, Binary Search)
- Sorting algorithms (Bubble, Selection, Insertion, Merge, Quick)
- Two-pointer technique (used in reverse, sum problems, sliding window)
- Sliding window problems
- Prefix sum and difference arrays
- Matrix problems (rotation, spiral, search in sorted matrix)
- Graph traversal (later — graphs are often stored as 2D arrays)

In other words: master arrays first, and every harder topic becomes easier.

13. Final Takeaway

Arrays teach you three core skills that every programmer needs:

- How to store many values in one place
- How to process those values systematically with loops
- How to think in terms of index → value relationships

If loops taught you repetition, arrays teach you structured problem solving. And when you combine loops + arrays + methods, you unlock the kind of logic that real programming relies on every day.

Key Takeaway: Arrays are not just "another topic" — they are the foundation. Get comfortable with indices, traversal, and the two-pointer pattern, and you will be ready for the next big topics: searching, sorting, and finally full-on DSA.

14. What's Next?

Once arrays feel comfortable, the natural next steps are:

- Strings — which are basically arrays of characters
- ArrayList — a smarter, resizable version of an array
- Searching algorithms (Linear Search and Binary Search)
- Sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort)
- Two-pointer and sliding window techniques
- Recursion with arrays